数字电子技术基础

庄梓伸

2022 年 4 月 13 日

目录

1 **数制与码制** 1

1.1 数制——进制数 1

1.2 码制 1

1.2.1 BCD 码 1

1.2.2 校验码 1

1.2.3 ASCII 码 1

1.3 二进制的数制问题解决 1

2 **逻辑代数基础** 3

2.1 逻辑运算 3

2.2 逻辑函数及其化简 3

2.2.1 逻辑函数 3

2.2.2 逻辑函数的化简 3

3 **数字电路分析与设计思路** 3

3.1 组合逻辑电路 3

3.1.1 组合逻辑电路的分析 3

3.1.2 组合逻辑电路的设计思路 4

3.2 时序逻辑电路 4

4 **组合逻辑电路的电路模块与应用** 4

4.1 编码器 4

4.2 译码器 8

4.3 数据选择器 8

4.4 数值比较器 8

4.5 加法器 8

1 数制与码制

1.1 数制——进制数

1.2 码制

1.2.1 BCD 码

8421BCD	余三码	BCD	BCD
0000	0011	243	34

表 1: BCD 码

1.2.2 校验码

1.2.3 ASCII 码

1.3 二进制的数制问题解决

下面将通过计算机对数的处理方式进行讲述，我们知道计算机用二进制的方式储存我们的十进制数，在计算机中一个字节 (bytes) 表示八个 bits，每个 bit 用 0 或 1 表示，这样的话一个字节就能表示 256 个数，我们将能表示的数的范围称为**模**。如果我们只需要表示正数，那我们只需将正的十进制数逐一用二进制表示即可，但我们出现了负数的概念，那么多出来了的负号要如何表示呢？下面我们先介绍负数的三种表示方式——原码、反码和补码。

原码的表示方式 我们用第 8 位，也即最高位作为符号位，0 表示正数，1 表示负数。在这样表示后，我们发现原本表示 0 255 的数制变成了-127~127。是的，我们可以发现这里仅有 255 个数，为什么呢？因为再当 0 作为正数时，其二进制表示为 0000 0000，而 0 作为负数时，其二进制表示为 1000 0000。所以这是这种表示方式的第一种问题：存在正负零。

然后我们再研究原码的运算，如果我们研究 $1+(-1)$ 这个问题，利用原码我们要怎么操作呢，首先我们要将-1 的符号位符号化为减法运算（事实上，计算机没有减法运算，此处只是为了更好理解，假设出来的），用 1(0000

0001) 减去 1(0000 0001))，再加上符号 (1000 0000)，得到最终答案 1000 0000(-0)。所以，这里又出现了原码的第二个问题：运算复杂。

反码的表示方式 为了解决原码运算复杂的问题，以及计算机无减法运算的问题，我们舍弃了符号位这一表示方法，进而引入一个新的概念——反码，什么是反码呢？将原码的每一位取反 (01 互换)，我们用得到的这个数表示其相对应的负数。例如-5 的反码就是 5(0000 0101) 每一位取反即 1111 1010。此时，我们再研究 $5+(-5)$ 这个问题，利用反码我们的计算机又要怎么操作呢？5(0000 0101) 直接与-5(1111 1010) 相加得到 1111 1111，这个值刚好为-0。这样一来，我们似乎既规避掉了计算机没有减法运算的问题，又完美解决了原码计算复杂的问题。但这种表示方法还存在两个问题需要解决：正负零问题；溢出问题。

正负零问题是先前便存在的。但新出现的溢出问题是什么呢？我们来看 $2-1$ 是怎样计算的呢？由于计算机没有减法运算，我们将这一式子化为 $2+(-1)$，也就是 2(0000 0010)+(-1)(1111 1110)，相加可得 1 0000 0000，我们发现数据出现第九位了，我们称这种情况为溢出，在反码运算中，我们必须让溢出位加进结果的最低位，也即 0000 0001，这样才能保证反码运算的正确性。而这一操作也给计算机带来了一定的麻烦。

补码的表示方法 那是否存在有一种既能解决正负零又能让计算机运算方便的表示方式呢？答案当然是有的——补码。类比时钟的运转，我们将超出模最大值的数继续用负数表示，也即用 128∼255 分别表示-128∼-1。举一个例子：-6 的补码表示为模 256-6=250(11111010)。这样一来我们就将原本前后不连续的数，连接了起来，也解决了正负零的问题。

然后我们研究一下要如何进行补码计算呢？

$$6-4=6+(-4)=6+(256-4)=6+252=00000110+11111100=00000010=2$$

我们惊奇的发现我们只需要将负数的补码表示与正数的原码表示相加，溢出位舍去我们就可以得到正确的答案。

那么计算机又是如何处理补码这个概念的呢？通过大量计算与经验总结，我们可以得到将负数取绝对值得到的正数的原码取反然后加 1，可以得到负数的补码。接下来我们以计算机的思路处理一遍负数：当计算机获取到-x 时，检测到负号，计算机进行如下操作：取其后的正值 x；将正值的原码逐位取反然后加 1，这样就得到了原本-x 的补码了。

至此，我们通过补码表示法也完美的解决了负数在计算机中的所有问题。

2 逻辑代数基础

2.1 逻辑运算

2.2 逻辑函数及其化简

2.2.1 逻辑函数

2.2.2 逻辑函数的化简

1. 代数法
2. 卡诺图

    不含无关项的卡诺图

    含有无关项的卡诺图

3 数字电路分析与设计思路

我们根据数字系统的逻辑功能和电路结构，将数字电路分为：组合逻辑电路和时序逻辑电路。

3.1 组合逻辑电路

3.1.1 组合逻辑电路的分析

1.

3.1.2 组合逻辑电路的设计思路

3.2 时序逻辑电路

4 组合逻辑电路的电路模块与应用

4.1 编码器

将输入的有效信号，转换为电平信号的形式，再产生二进制代码与其对应的逻辑电路模块。

1. 二进制编码器

   用 n 位二进制代码对 2^n 个信号进行编码的电路

   普通编码器

   我们常用的普通编码器有 8 线-3 线编码器，也即有 8 个输入端，3 个输出端的电路。我们规定在任意时刻，其输入仅有一个取值为 1，且其输入与输出一一对应。其真值表如下：

输入								输出		
A0	A1	A2	A3	A4	A5	A6	A7	B2	B1	B0
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

图 1: 8 线-3 线编码器真值表

   根据真值表我们可以写出 B_2、B_2、B_2 的表达式，举例 B_2 的一个最小项如下：

$$\overline{A_0} \cdot \overline{A_1} \cdot \overline{A_2} \cdot \overline{A_3} \cdot A_4 \cdot \overline{A_5} \cdot \overline{A_6} \cdot \overline{A_7}$$

   该最小项与其他无关项合并最后可得

$$A_4$$

同理，我们可以得到

$$B_2 = A_4 + A_5 + A_6 + A_7 \tag{1}$$

同理，我们又可得另外两个输出，

$$B_1 = A_2 + A_3 + A_6 + A_7 \tag{2}$$

$$B_0 = A_1 + A_3 + A_5 + A_7 \tag{3}$$

其逻辑电路如下：

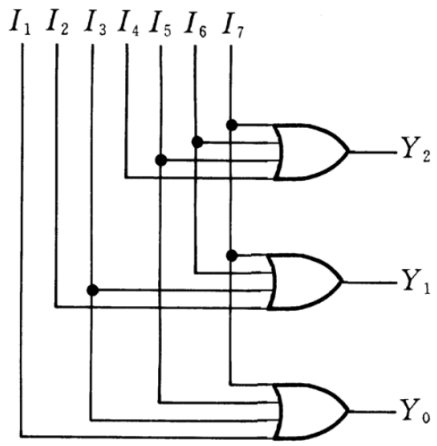

图 2: 8 线-3 线编码器逻辑电路

优先编码器

上述的普通编码器虽然达到了编码控制电路的目的，但是任意时刻仅允许一个输入信号有效。然而对于我们常见的应用场景，我们往往需要多个有效输入信号，因此我们通过给所有信号赋予优先顺序，当多个信号有效时，仅对优先级最高的信号进行编码。于是，我们将按优先级从高至低排列，有以下编码方式：

输 入								输 出		
$\overline{I}_0$	$\overline{I}_1$	$\overline{I}_2$	$\overline{I}_3$	$\overline{I}_4$	$\overline{I}_5$	$\overline{I}_6$	$\overline{I}_7$	$\overline{Y}_2$	$\overline{Y}_1$	$\overline{Y}_0$
x	x	x	x	x	x	x	x	1	1	1
x	x	x	x	x	x	x	0	0	0	0
x	x	x	x	x	x	0	1	0	0	1
x	x	x	x	x	0	1	1	0	1	0
x	x	x	x	0	1	1	1	0	1	1
x	x	x	0	1	1	1	1	1	0	0
x	x	0	1	1	1	1	1	1	0	1
x	0	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1

图 3: 引入优先级编码方式

从真值表我们可以看出有三组不同的输入有相同的输出，不符合编码器的输入输出唯一性，实际使用会产生异常。因此，我们再引入一个输入端口，称为使能输入端，该端口输入的信号称为选通信号，记为 S(select)。

我们将允许多个有效信号输入的编码器称为优先编码器。我们常用的 74LS148 就是一种 8 线-3 线优先编码器，其真值表如下：

输入									输出				
$\overline{S}$	$\overline{I_0}$	$\overline{I_1}$	$\overline{I_2}$	$\overline{I_3}$	$\overline{I_4}$	$\overline{I_5}$	$\overline{I_6}$	$\overline{I_7}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$	$\overline{Y_s}$	$\overline{Y_{ex}}$
1	x	x	x	x	x	x	x	x	1	1	1	1	1
0	x	x	x	x	x	x	x	0	0	0	0	1	0
0	x	x	x	x	x	x	0	1	0	0	1	1	0
0	x	x	x	x	x	0	1	1	0	1	0	1	0
0	x	x	x	x	0	1	1	1	0	1	1	1	0
0	x	x	x	0	1	1	1	1	1	0	0	1	0
0	x	x	0	1	1	1	1	1	1	0	1	1	0
0	x	0	1	1	1	1	1	1	1	1	0	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	0
0	1	1	1	1	1	1	1	1	1	1	1	0	1

知乎 @随风

图 4: 74LS148 真值表

其逻辑图如下：

从真值表我们可以得到 74LS148 的输入输出端都是**低电平有效**，且可以得到以下表达式：

$$\begin{aligned} F_2 &= S \cdot I_7 + S \cdot I_6 \cdot \overline{I_7} + S \cdot I_5 \cdot \overline{I_6} \cdot \overline{I_7} + S \cdot I_4 \cdot \overline{I_5} \cdot \overline{I_6} \cdot \overline{I_7} \\ &= S \cdot (I_7 + I_6 + I_5 + I_4) \end{aligned} \tag{4}$$

可以看出，**当选通信号 $\overline{S} = 0$ 时，其他输入可以正常工作；当 $\overline{S} = 1$ 时，其他输出被封锁，输出均为高电平。**

从逻辑图可以得到 F_S 和 $\overline{F_{EX}}$ 的表达式：

$$\begin{aligned} F_S &= I_0 + I_1 + I_2 + ... + I_7 + \overline{S} \\ &= \overline{\overline{I_0} \cdot \overline{I_1} \cdot \overline{I_2} \cdot ... \cdot \overline{I_7} \cdot S} \\ &= \overline{\overline{I_0 + I_1 + I_2 + ... + I_7} \cdot S} \end{aligned} \tag{5}$$

$$\begin{aligned} \overline{F_{EX}} &= \overline{F_S \cdot S} \\ &= \overline{S} + S \cdot \overline{I_0 + I_1 + I_2 + ... + I_7} \\ &= \overline{S} + \overline{I_0 + I_1 + I_2 + ... + I_7} \\ &= \overline{S \cdot (I_0 + I_1 + I_2 + ... + I_7)} \end{aligned} \tag{6}$$

从 (5) 的表达式中，我们可以得知只有当 $\overline{S}=0$，也即其他输入正常工作，且不存在有效信号时，F_S 才输出为低电平。因此，F_S **输出低电平表示该电路正常工作，且无有效信号输入**。我们称 F_S 为**使能输出端**。

从 (6) 的表达式中，我们可以得知只有当 $\overline{S}=0$，也即其他输入正常工作，且存在有效信号时，F_{EX} 才输出为低电平。因此，F_{EX} **输出低电平表示该电路正常工作，且存在有效信号输入**。我们称 F_{EX} 为**扩展输出端**。

2. 二-十进制编码器

4.2 译码器

4.3 数据选择器

4.4 数值比较器

4.5 加法器